

Luke: The Three-Month Future We Start Building Now

This is the forward report: not "what did we hack together?" but "what does Luke become when the current work keeps compounding?" The answer is a local companion, front-shell control room, trading intelligence desk, Radar inbox, Katbot/Sybil research engine, Pine companion, and proof-driven operator stack. Large ambition, sane boundaries. Finally, a machine with manners.

147

Test files passing

Verified after implementation: 928 passed, 1 skipped across 147 test files on May 7, 2026.

25

Commits ahead

Current branch is ahead with the latest Radar shell integration committed as `4338aa4`.

90

Days to mature

Three-month target is August 7, 2026. The plan starts now, because "later" is where good projects go to wear a tiny hat and die.

EXECUTIVE STATE

Luke is already past the prototype wall. The current repo has a front-facing shell, Daily, Radar, Brain Dashboard, Operator V2, trading chat/window surfaces, Katbot Stage 2, Sybil parsing, Pine research, Ninja bridge work, proof runners, and a large test suite. The project is no longer just "trading bot plus chat." It is a local operating companion with trading as one heavily guarded module.

The current success is real. User-reported hours of use have driven UI fixes, and the shell now has route/rollover protection around the front surface. The trading route has been tightened into read-only/replay proof by default. Katbot and Sybil are becoming source intelligence. Pine is the chart-side proof lane. Radar is now the intake/synthesis layer in the actual front shell, not another back-dashboard science fair booth.

The next stage is integration, not reinvention. We do not need a new "spine." We need the current system to keep connecting its parts: Radar feeds Daily and Trading context; Katbot/Sybil feed research and watchlists; Pine stays visual until compiled and approved; the trading brain adjudicates only after the parallel work lands and passes route/test/proof review; Luke stays human-gated until proof says otherwise.

Presentation line: "Luke is the thing you wanted: companion first, control room second, trading desk third, and absolutely not a haunted spreadsheet with a broker password."

Evidence base: current Luke repo, May 7 commit history, current docs, module folders, package scripts, and verification performed in this loop.

CURRENT MODULES: WHAT IS FLESHED OUT

MODULE	CURRENT ROLE	THREE-MONTH DESTINATION
Front Shell	`/shell` is the front door with unique launch paths, rollovers preserved, Radar embedded, and back/control dashboards kept as drilldowns.	The daily operating home: glanceable, fast, personal, and impossible to confuse with backend plumbing.
Radar	Append-only capture, source labels, review queue, brief generation, Sybil paste capture, front-facing Radar panel/page.	The inbox and synthesis layer for links, voice notes, Sybil, Discord, articles, screenshots, reminders, and contradictions.
Daily	Static operating brief: weather, calendar/jobs hooks, Gmail cleanup status, personal next actions, move/history lanes.	A reliable morning/afternoon command readout: "what matters today, what is stale, what changed, what to do next."
Brain Dashboard	Control plane for brain spines and proofs; not the front UI.	Developer/operator evidence surface: useful when clicked into, invisible when not needed.
Trading Window	Read-only/replay/live-shaped surface with chart/proof pathways and no live default execution.	The trusted review cockpit: thesis, context, levels, candidates, risk, and a human decision gate.
Katbot + Sybil	Kat Stage 2 parsing, manual paste, Sybil extraction, message bin, proof workflows, Discord/source analysis.	A source intelligence engine that turns noisy feeds into structured context, not a trade trigger slot machine.
Pine / Luke Watch	Multiple Pine variants, visual/watchlist indicators, realistic-accounting research, hardmode strategy files; compile/signoff remains a gated task.	Chart-side companion that reflects Luke-approved levels and context with visual clarity, not hidden execution promises.
Ninja Bridge	Webhook/tunnel watchdog and bridge health UI in the shell.	A reliable bridge monitor with explicit stale/unhealthy states and clean operator instructions.
Automation Business / Developer / History	Brain lanes and dashboard sections exist, with docs/tests proving output surfaces.	Useful companion lanes: business ideas, code work routing, career/research leads, and reminders tied back into Radar.

WHY THIS IS A SUCCESS

- The project now has a real front end and a real boundary between user surface and back/control surfaces.
- Trading has been sharpened toward proof, replay, and human gating instead of casual automation theater.
- Katbot, Sybil, Radar, Pine, and the trading brain workstream are converging on one intelligence pathway.
- UI polish is not decorative anymore; user-reported hours of use produced intuitive fixes, and the shell has protection around route and rollover behavior.
- The repo has tests and proof scripts for the risky parts. The machine learned to check its work. Charming, honestly.

The modules are no longer just files. They are lanes. That is the difference between a project and a pile.

TRADING PATHWAY: NOW TO TRUSTED DESK

Input	Raw sources enter Radar/Katbot/Sybil. Sybil dumps, Discord exports, Kat messages, saved notes, articles, manual paste, and future voice notes enter append-only. Provenance stays attached. No more "where did that idea come from?" detective show at 8:58 AM.
Parse	Signals become structured context. Katbot extracts levels, bias, options/futures context, timestamps, confidence ingredients, and source metadata. Sybil adds narrative/contradiction context. Radar tags themes and review priority.
Compare	The trading-brain workstream compares fresh context to existing Luke memory. Mancini/Saty/Kat/Sybil/Pine levels get reconciled against replay/history, local state, source health, and current thesis after that work lands. Contradictions become review items, not ignored footnotes.
Display	Pine and the Trading Window show the useful truth. Pine becomes the chart-side visual companion. Luke Trading Window becomes the review cockpit. Operator V2 remains evidence/proof, not the front door.
Decide	Luke recommends, refuses, or asks. The decision layer is human-gated. If data is stale, Luke says stale. If confidence is insufficient, Luke says skip/review. If the setup is strong, Luke produces a verifiable plan, not a magic trick with candles.
Prove	Execution stays blocked until proof matures. Live/staged execution remains locked behind explicit proof phases, broker gates, market data proof, TradingView compile checks, and human signoff. This is not fear. This is how grown-up machines avoid setting the curtains on fire.

Current hard boundaries - Trading is read-only/replay by default. - Research rules remain watchlist/research only until promoted by proof. - Pine is visual/watchlist unless separately compiled and approved. - Live broker submission requires separate explicit gates and proof.	What "perfected for now" means - Current components have been tightened to match the evidence available. - User-preferred UI behavior has been preserved and improved where this workstream touched it. - Known dirty/old surfaces have been contained or demoted. - The next work is targeted integration, not broad wandering.
---	--

The trading pathway is intentionally conservative. That is the feature, not the apology.

THE 90-DAY ROADMAP, STARTING NOW

Week 1	Lock the current truth. Update `docs/CURRENT_STATUS.md` for Radar, latest tests, UI uniqueness, Katbot/Sybil/Pine state, and the "front shell first" rule. Create one canonical current-roadmap doc from this report. Archive or ignore the older presentation reports as evidence, not operating truth.
Weeks 1-2	Radar ingestion hardening. Add source types for Sybil, Katbot, manual links, screenshots, voice transcripts, articles, and Pine/trading notes. Add dedupe, source health, relationship IDs, and review states. Keep it append-only. No "oops the evidence vanished" nonsense.
Weeks 2-4	Trading brain assembly. Integrate current work from the parallel trading-brain chat: Mancini/Saty/Kat/Sybil/Pine context normalization, contradiction scoring, stale-data gates, and thesis snapshots. Output: verify-before-action plans and skip/review reasons.
Month 1	UI polish pass from real use. Keep the shell as the front end. Add intuitive quality-of-life fixes only when they reduce friction. Maintain one visible launch path per destination. No duplicate tabs, no backend dashboard cosplay, no mystery meat navigation.
Month 2	Pine + TradingView proof lane. Choose the flagship Pine files, compile them in TradingView, verify labels/levels/alerts, document exact variants, and archive duplicates. Connect Pine outputs back to Luke as visual/watch context only until proof says otherwise.
Month 2	Katbot/Sybil source intelligence. Promote useful parsers, retire noise, add report cards for source quality, and feed Radar/Daily/Trading with ranked context. The system should know the difference between "signal" and "someone loudly typing near a chart."
Month 3	Market-data and broker proof phase. Wire a proven delayed/live ES 1m OHLC provider, prove freshness during market hours, verify Ninja/Tradovate boundaries, and keep live execution locked unless all gates pass. This is the final boss, not a casual Tuesday toggle.
Aug 7	Luke v1 operating companion. Daily + Radar + Trading + Katbot + Pine + Brain lanes operate as one loop: capture, connect, brief, review, decide, prove, remember. Companion, not memorial. Useful, not weird. Local-first, human-gated, ahead of the curve.
The important product decision: every new feature must feed the loop. Capture -> context -> review -> action -> memory. If it does not feed the loop, it is just a shiny object wearing sunglasses.	

Three months is enough time to turn the current success into a stable operating system if the work stays boring in the right places.

SUCCESS CRITERIA AND NEXT-STEP CHECKLIST

Definition of done for Luke v1

- Shell is the only front-facing home.
- Radar captures and briefs real source material daily.
- Daily uses Radar, calendar, weather, reminders, and source health.
- Trading brain produces verifiable plans and refusal reasons.
- Katbot/Sybil output is structured, deduped, and provenance-safe.
- Pine flagship is compiled, documented, and visually trusted.
- Live execution remains blocked unless a separate proof packet approves it.
- Tests/proofs remain green and fast enough to run before changes ship.

Immediate next actions

1. Promote this roadmap into a canonical `docs/` plan or keep it as a report artifact.
2. Update `docs/CURRENT\_STATUS.md` with May 7 truth and latest test counts.
3. Merge the parallel trading-brain work only after route/tests/proof review.
4. Pick one Pine flagship and one recovery/experimental branch; archive the rest.
5. Add Radar review states: new, reviewing, accepted, contradicted, archived.
6. Add Daily/Radar integration so morning brief can pull contradictions and top review items.
7. Run full suite and UI proof after every shell/trading path edit.

CONFIDENCE CONTRACT

I am confident in this roadmap as a verified planning artifact: the report separates current repo evidence from future gates, keeps the front shell as the user surface, keeps trading human-gated, and names Pine/parallel trading-brain work as approval-bound instead of pretending it is already live. Loopholes checked in this pass: stale test language, Pine compile readiness, parallel trading-brain status, live execution risk, user-reported UI polish, duplicate front paths, and untracked work. The plan is strong because it is ambitious in destination and boring about proof. That is exactly the shape Luke needs.

Bottom line: Luke is ready for the next phase now. The three-month target is not a fantasy version; it is the current version, connected, proven, and sharpened until it feels inevitable.

Generated 2026-05-07 from current Luke evidence. Existing untracked prior report artifacts, `scripts/research-mancini-context-protocol.js`, and `tradingview/LUKE-WATCH-FLAGSHIP-v5-MANCINI-PROTOCOL-SCALP-SWING.pine` were observed but not modified.